SE 4010 – Current Trends in Software Engineering [2026/JAN]

Assessment 01

Cloud Computing Assignment Report

Submitted by: R26-SE-040

Semester 1

Registration number	Name	Service
IT22272522	**Ashandth Uthayashankar**	**User Service**
IT22203694	**Sobiya Anton Suresh**	**Product Service**
IT21816086	**S S Y Wickramasinghe**	**Order Service**

Contents

1. Introduction

This project presents the comprehensive design, development, and deployment of a secure, cloud-based microservice application. The primary objective of this assignment is to move away from monolithic architectures by developing a set of independently deployable microservices that integrate seamlessly to form a cohesive, end-to-end working application.

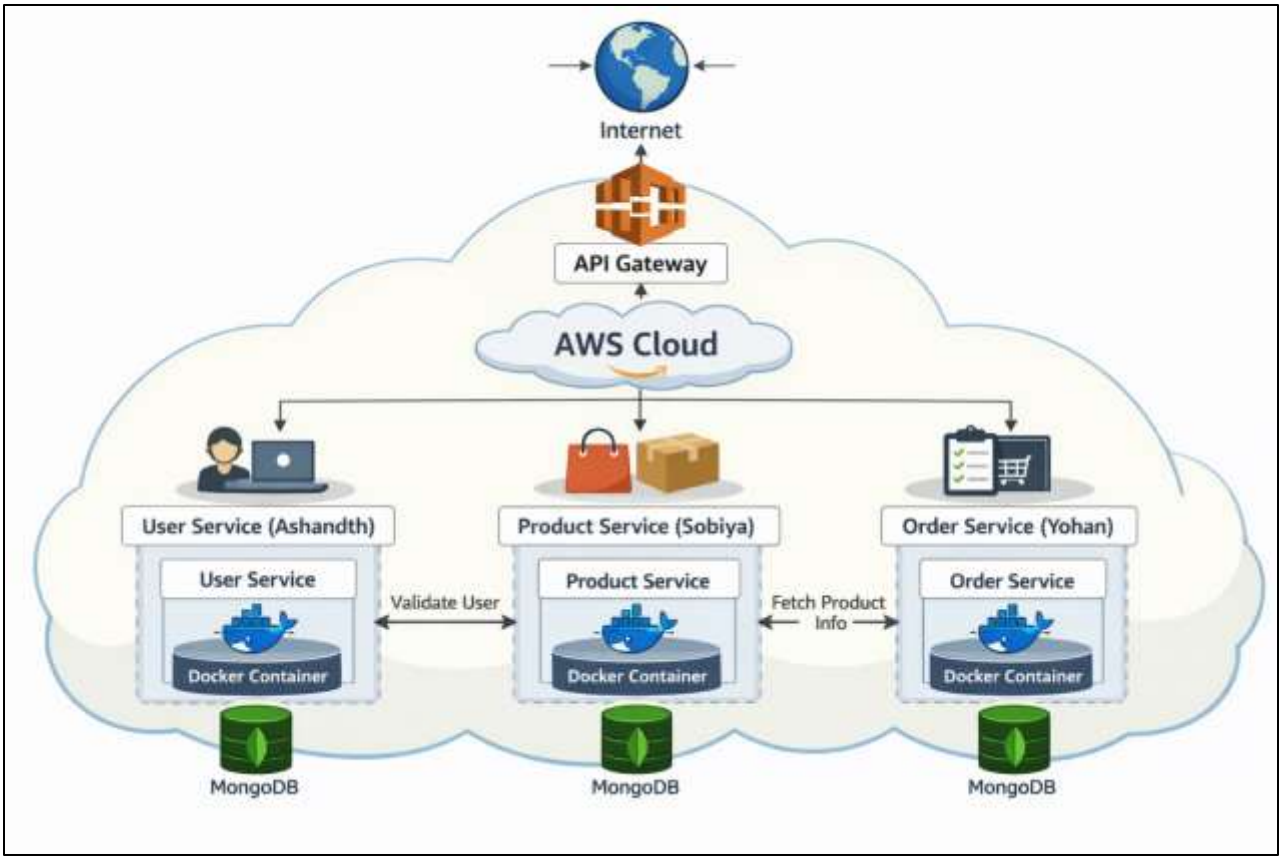
The chosen domain is a **Microservice-Based E-Commerce Application**. This system allows users to securely register accounts, browse product catalogs, and place orders. To achieve this, the system is decomposed into three highly cohesive but loosely coupled microservices: the User Service, Product Service, and Order Service. By leveraging modern cloud capabilities, Docker containerization, and automated CI/CD pipelines, this project demonstrates a complete software development lifecycle that adheres to industry-standard DevOps and DevSecOps practices.

2. Shared Architecture & Infrastructure Design

The system follows a modular microservices architecture, ensuring that the failure of one component does not cascade and bring down the entire application.

Architectural Principles:

- **Independent Containerization:** Every microservice is packaged into its own Docker container, guaranteeing that the application runs identically across local development and production environments.
- **Database Isolation:** To maintain loose coupling, each service manages its own MongoDB database instance. This prevents services from bypassing APIs to read another service's data directly.
- **Synchronous Communication:** Services communicate with one another over the network via standard HTTP REST APIs using Axios payloads.
- **Cloud Infrastructure:** The services are hosted on Vercel as three independent projects, utilizing cloud resources while remaining strictly within the Free Tier limitations.



3. Microservice Specifications

3.1 User Service (Ashandth)

- **Rationale & Role:** The User Service acts as the identity provider for the e-commerce system. It is responsible for safely storing user credentials and issuing authorization tokens.
- **Core Functionality:**
  - **Registration:** Accepts user details, hashes passwords securely using bcrypt before storing them in MongoDB, and creates the user profile.
  - **Authentication:** Verifies login credentials and generates a stateless JSON Web Token (JWT) with an expiration window.
  - **Identity Verification:** Provides an endpoint for other services to validate if a user ID exists and is active.

- **Endpoints & API Contract:**
  - POST /api/register (Payload: Name, Email, Password)
  - POST /api/login (Payload: Email, Password -> Returns JWT)
  - GET /api/user/{id} (Returns sanitized user profile)
- **Technology Stack:** Node.js, Express.js, MongoDB, JWT Authentication.

3.2 Product Service (Sobiya)

- **Rationale & Role:** The Product Service manages the inventory and catalog. It acts as the single source of truth for item pricing, descriptions, and stock availability.
- **Core Functionality:**
  - **Catalog Management:** Allows administrators to add new products to the database.
  - **Product Browsing:** Retrieves the full list of available products for the storefront front-end.
  - **Detail Retrieval:** Fetches specific product data required during the checkout process.
- **Endpoints & API Contract:**
  - POST /api/products (Payload: Name, Description, Price, Stock)
  - GET /api/products (Returns array of product objects)
  - GET /api/products/{id} (Returns single product details)
- **Technology Stack:** Node.js, Express.js, MongoDB.

3.3 Order Service (Yohan)

- **Rationale & Role:** The Order Service orchestrates the checkout process. It does not store user or product data itself; instead, it aggregates data from the other services to finalize a transaction.
- **Core Functionality:**
  - **Order Placement:** Receives a request containing a User ID and an array of Product IDs.
  - **Cross-Service Validation:** Communicates with the User and Product services to verify the payload before generating an order ticket.

- **Endpoints & API Contract:**
    - POST /api/order (Payload: UserID, Array of ProductIDs)
- **Technology Stack:** Node.js, Express.js, MongoDB, Axios.

4. Inter-Service Communication & Integration

A core requirement of this architecture is that every microservice must communicate with at least one other service in the group. We implemented this through a synchronous REST API integration pattern.

Detailed Integration Flow (Placing an Order):

1. **Request Initiation:** A client sends a POST /api/order request containing { "userId": "123", "productId": "456" }.
2. **User Validation (Integration 1):** The Order Service executes an HTTP GET request to the User Service (GET /api/user/123). If the User Service returns a 404 Not Found, the order is aborted.
3. **Product Validation (Integration 2):** Concurrently, the Order Service executes an HTTP GET request to the Product Service (GET /api/products/456). The Product Service returns the item's price and validates its existence.
4. **Transaction Completion:** Upon receiving 200 OK responses from both external services, the Order Service calculates the total, saves the order record to its own MongoDB database, and returns a success response to the client.

5. DevOps Engineering Practices

To ensure consistent delivery and deployment, we automated our workflows using fundamental DevOps practices.

- **Version Control:** All source code is maintained in public GitHub repositories, facilitating collaborative development and version tracking.
- **Containerization Lifecycle:** * We authored a multistage Dockerfile for each microservice.
    - During the build phase, dependencies are installed and the application is packaged.
    - The built images are automatically pushed to Docker Hub, our chosen container registry.

- **CI/CD Automation (GitHub Actions):** We developed a YAML workflow file that triggers on every push to the main branch. The pipeline automatically lints the code, executes the Docker build sequence, pushes the image to Docker Hub, and triggers a webhook.
- **Cloud Deployment:** We utilized Vercel to host our application. Vercel provisions the necessary cloud infrastructure to expose our APIs over the internet securely, providing a robust environment for our Node.js services.

6. Security & DevSecOps Implementation

Security was integrated into our pipeline from day one, adhering to the "Shift-Left" DevSecOps philosophy.

- **Static Application Security Testing (SAST):** We integrated free managed SAST tools (SonarCloud / Snyk) directly into our GitHub repositories. These tools automatically scan pull requests for common vulnerabilities, exposed secrets, and code smells before merging.
- **Identity & Access Management:** We utilized JWTs for stateless authentication. Passwords are never stored in plaintext.
- **Principle of Least Privilege:** Database users were created with scoped permissions restricted strictly to their corresponding microservice databases.
- **Environment Configuration:** All sensitive configuration data, such as MongoDB connection strings and API keys, are injected at runtime using environment variables. This ensures no secrets are leaked in the public GitHub repositories.

7. Challenges Encountered & Resolutions

Throughout the development lifecycle, we encountered and overcame several technical hurdles:

1. **Service Discovery and Environment Routing:** Initially, hardcoding localhost URLs in the Order Service caused failures when deployed to the cloud. *Resolution:* We implemented dynamic environment variables (USER_SERVICE_URL, PRODUCT_SERVICE_URL) that automatically route traffic to local ports during development and to live Vercel domains during production.
2. **Container Configuration on Vercel:** Adapting standard Docker-based Node.js applications to deploy correctly within Vercel's specific project structure required troubleshooting. *Resolution:* We refined our package.json build scripts and adjusted Vercel's root directory settings to properly serve the Express backend.

3. **Cross-Origin Resource Sharing (CORS) Blocks:** When integrating the front-end client with the various cloud-deployed microservices, browser CORS policies blocked the requests. *Resolution:* We properly configured the cors middleware in Express to accept requests from authorized origins.
4. **Asynchronous Validation Timing:** When the Order Service queried the User and Product services sequentially, it increased response latency. *Resolution:* We refactored the Axios calls to use Promise.all(), allowing the Order Service to fetch user and product validations simultaneously, cutting the wait time in half.

8. Code Repositories & Live Deployment Links

Below are the links to the public GitHub repositories containing the source code, CI/CD pipeline configurations, and Dockerfiles for each microservice, alongside their live deployment URLs on Vercel.

User Service (Ashandth)

- **GitHub Repository:** Click here to see the github repository
- **Live Vercel URL:** Towards user service
- **API Documentation:** Towards user service api docs

Product Service (Sobiya)

- **GitHub Repository:** Click here to see the github repository
- **Live Vercel URL:** Towards product service
- **API Documentation:** Towards product service api docs

Order Service (Yohan)

- **GitHub Repository:** Click here to see the github repository
- **Live Vercel URL:** Towards order service
- **API Documentation:** Towards order service api docs

9. Conclusion

This project successfully fulfills the objective of designing and implementing a secure, microservice-based application. By decomposing an e-commerce platform into three distinct services, utilizing Docker for containerization, and establishing automated CI/CD pipelines, our group demonstrated a practical understanding of modern cloud computing and DevSecOps principles.

10. References

[1] OpenJS Foundation, "Node.js Documentation," *Node.js*. [Online]. Available: https://nodejs.org/en/docs/. [Accessed: Mar. 22, 2026].

[2] Docker Inc., "Docker Documentation," *Docker*. [Online]. Available: https://docs.docker.com/. [Accessed: Mar. 22, 2026].

[3] GitHub Inc., "GitHub Actions Documentation," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/actions. [Accessed: Mar. 22, 2026].

[4] MongoDB Inc., "MongoDB Documentation," *MongoDB*. [Online]. Available: https://www.mongodb.com/docs/. [Accessed: Mar. 22, 2026].

[5] SmartBear Software, "Swagger Documentation," *Swagger*. [Online]. Available: https://swagger.io/docs/. [Accessed: Mar. 22, 2026].

[6] SonarSource S.A., "SonarCloud Documentation," *SonarCloud*. [Online]. Available: https://docs.sonarcloud.io/. [Accessed: Mar. 22, 2026].

[7] Snyk Ltd., "Snyk Documentation," *Snyk*. [Online]. Available: https://docs.snyk.io/. [Accessed: Mar. 22, 2026].